Impact Report

Baseline vs. Agentic Pipeline Results, and Tool-Evolution Drill Summary

Executive Summary

This report compares a manual human review baseline against the agentic PR review pipeline, using the same seeded-bug ground-truth methodology throughout, and summarizes a real governance drill run against the pipeline's safety controls. All numbers below are drawn from actual, captured test runs — none are estimated or projected unless explicitly labeled as such.

Headline result: across the full holdout set, the pipeline caught the one seeded bug present with a grounded, specific explanation, and correctly approved both clean PRs with zero false positives. The deterministic conversion built specifically to fix the baseline's weakest area (inconsistent severity framing) held up as intended in this run. A deliberately-introduced governance regression was caught automatically before it could reach production.

Methodology

Baseline: a human reviewer manually reviewed 5 real, merged pull requests from chalk/chalk (3 modified to include one intentional, documented bug each; 2 left clean), timing each review and self-scoring against a 5-dimension quality rubric.

Agentic pipeline: the same evaluation approach was applied to a held-out set of PRs (deliberately not used to build or tune the pipeline — see the calibration/holdout split), with Claude performing the reviewer agent's reasoning directly against each diff, combined with the pipeline's real deterministic scoring code and real automated eval checks. This differs from a fully automated API-driven run in one respect — see the Limitations section — but the judgment step itself is genuine, not simulated or hand-crafted to fit an expected answer.

1. Baseline Results (Pre-Agent)

Metric	Result
Bug detection rate	3 / 3 (100%)
Review latency	5 min average per PR (self-timed)
Quality (rubric score)	75% average (83% seeded-bug PRs, 62.5% clean PRs)
Cost per review	$6.25 (5 min × $75/hr proxy rate)
Known weakness	Risk Flagging scored 1/2 consistently — correctly identified issues, inconsistent on stating severity/action

2. Agentic Pipeline Results (Holdout Set)

PR	Type	Finding	Recommendation	Eval Result
pr-4179	Seeded bug	Correctly located the exact missing-character break, with grounded reasoning	request_changes	PASS (4/4 checks)

PR	Type	Finding	Recommendation	Eval Result
pr-653	Clean	One real, low-severity note (missing test coverage) — correctly not escalated	approve	PASS (4/4 checks)
pr-3728	Clean	No findings — genuinely nothing to flag	approve	PASS (4/4 checks)

All 3 holdout PRs passed every automated check in `evals/run_eval.py`: schema validity, grounding (every finding traces to a real line in the diff), seeded-bug detection where applicable, and zero false positives on clean PRs.

3. Baseline vs. Agentic: Side-by-Side

Metric	Baseline (Human)	Agentic Pipeline	Change
Bug detection rate	3/3 (100%)	1/1 in holdout (100%)	Matched (smaller sample)
False positives on clean PRs	0/2	0/2	Matched
Risk Flagging consistency	Inconsistent (1/2 across all cases)	Structurally guaranteed by deterministic scoring code (ADR-001)	Improved by design — the specific weakness this conversion targeted
Review latency	5 min (timed)	Not directly comparable this run — conversational reasoning, not automated timing	Not measured this way
Cost per review	$6.25 (proxy)	$0 in this run (no billed API calls made)	Not a fair comparison as-is

4. Deterministic Conversion Impact

The risk-scoring step (mapping a reviewer's findings to an overall recommendation) was converted from agent-invoked reasoning to deterministic code, specifically to address the baseline's Risk Flagging weakness. Full reasoning and rejected alternatives are in ADR-001; the measured impact:

Metric	Before (estimated)	After (measured)
Latency per call	Typical LLM call range (not timed in this project)	0.0011 ms/call
Cost per call	Non-zero (API tokens)	$0
Consistency	Not guaranteed run-to-run	Guaranteed — 9/9 rule-branch tests pass every run

5. Tool-Evolution Drill Summary

A permission was deliberately revoked in the storage server's code (removing the reviewer agent's calibration-log read access) to test whether the pipeline's safety net catches a realistic, accidental regression — not a hypothetical.

Stage	What Happened
Before	5/5 policy tests pass (baseline state)
Change introduced	reviewer's calibration-log grant removed from TOOL_GRANTS in mcp/storage_server.py, without updating the governance policy doc
Caught by	tests/test_policy.py — 4/5 pass, 1 fails with an exact, specific diff between actual and expected permissions
Real breakage confirmed	A live call to the affected tool from the reviewer role returned a genuine PermissionError — not a silent failure
Fix & recovery	Permission restored; 5/5 policy tests pass again; live tool call confirmed working

Why this matters: this is the realistic failure mode — an accidental permission drift during normal development — not a deliberate attack. The CI-enforced policy test caught it before it could reach a merged pull request, with a specific, actionable error message naming exactly which role and tool grant diverged.

6. Honest Limitations

- **Small sample:** the holdout set contains only 1 seeded-bug PR (vs. 3 in the baseline). This is directional evidence of the pipeline working correctly, not a statistically strong claim.
- **Latency/cost not directly comparable:** this run used Claude reasoning directly in conversation rather than orchestrator.py's automated API call path, so timing and token-cost figures aren't measured on the same basis as the baseline's stopwatch-timed review. A fully automated run with a live API key would produce genuinely comparable numbers.
- **Rubric scoring not yet applied:** the automated eval checks are deterministic pass/fail, not the 0-2 scale used to score the human baseline. Applying the full 5-dimension rubric to these 3 real outputs would allow a true like-for-like quality score comparison.

Full methodology, raw outputs, and source data: docs/impact-study.md, docs/tool-evolution-drill.md, docs/baseline-run-notes.md, evals/sample-runs/.